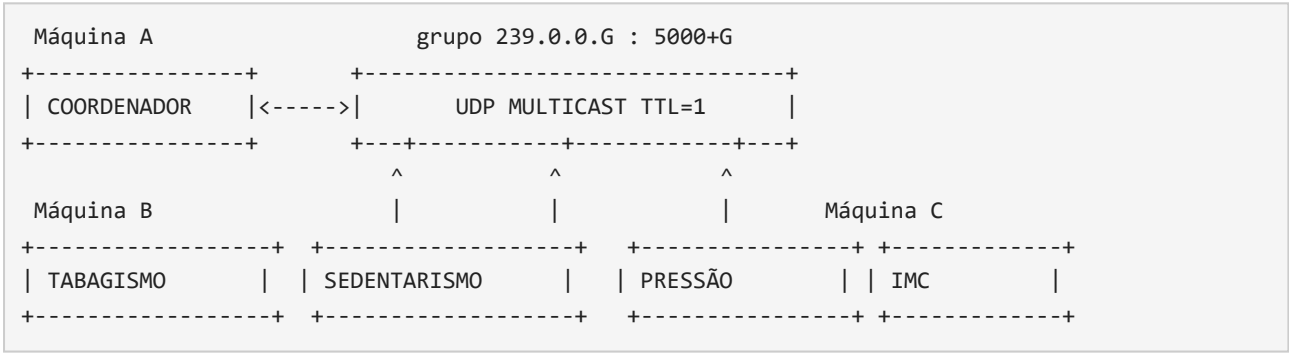
Sistema distribuído de avaliação didática de risco cardiológico com LPA2v

Integrantes: Evandro, Joel, Lucas, Arhur Alves e Iago
Grupo: 4
Tecnologia: Java 17, UDP multicast e Lógica Paraconsistente Anotada de 2 Valores

> O sistema e suas escalas são exclusivamente didáticos. Não constituem um instrumento clínico e não devem ser usados para avaliar pessoas reais.

1. Visão geral e arquitetura

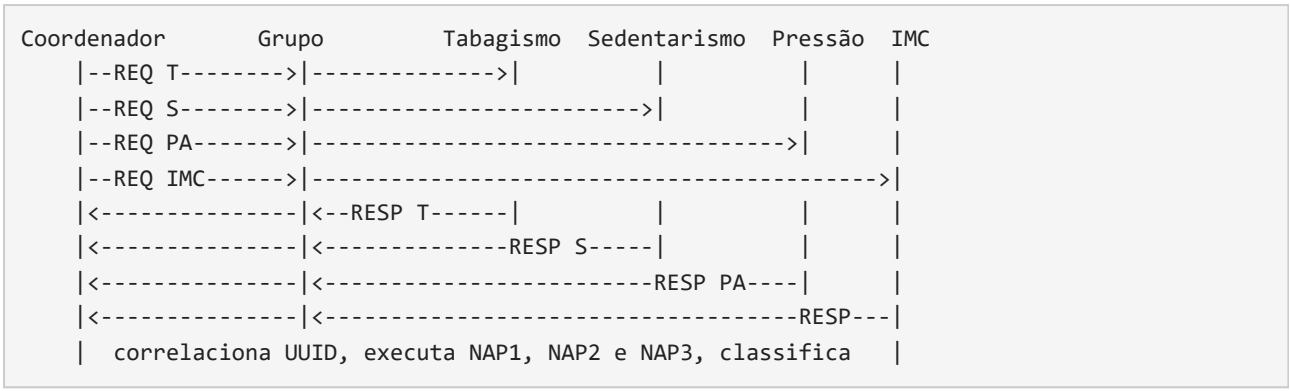
O sistema possui cinco processos Java independentes. O coordenador coleta os questionários e medidas, cria um UUID e envia quatro requisições. Cada agente escuta o mesmo grupo multicast, descarta mensagens alheias, calcula somente seu fator e publica uma resposta no próprio grupo. Não há TCP, RMI nem UDP unicast.



Todos ingressam com `joinGroup(SocketAddress, NetworkInterface)`. A interface é obrigatoriamente indicada por nome ou IPv4. O socket usa `setTimeToLive(1)`, de modo que os datagramas ficam restritos à rede local.

2. Protocolo e sequência

Cada datagrama UTF-8 possui cinco campos separados por ponto e vírgula: `TIPO;ID;ORIGEM;DESTINO;DADOS`. Os tipos são `REQ`, `RESP` e `ERRO`. O UUID correlaciona respostas mesmo com dois coordenadores simultâneos.



Exemplos: `REQ;a3f9c2;COORD;PRESSAO;150,95`, `RESP;a3f9c2;PRESSAO;COORD;0.500` e `ERRO;a3f9c2;IMC;COORD;altura fora da faixa plausível`.

3. Implementação

3.1 Ingresso, saída e TTL

O canal cria um socket reutilizável, vincula-o à porta comum, seleciona a interface e ingressa no grupo. A reutilização permite vários processos na mesma máquina. O encerramento sai do grupo antes de fechar o socket.

```
socket = new MulticastSocket(null);
socket.setReuseAddress(true);
socket.bind(new InetSocketAddress(config.porta()));
socket.setNetworkInterface(interfaceRede);
socket.setTimeToLive(1);
socket.joinGroup(enderecoGrupo, interfaceRede);

// no encerramento
socket.leaveGroup(enderecoGrupo, interfaceRede);
socket.close();
```

3.2 Recepção, filtragem e registro

Cada datagrama recebido é registrado antes da interpretação. O agente só aceita `REQ` de `COORD` cujo destino coincide com seu tipo. Mensagens próprias, alheias, malformadas ou duplicadas são descartadas e também registradas.

```
Mensagem mensagem = Mensagem.interpretar(texto);
if (mensagem.origem().equals(tipo.name())) {
    log.descartada("mensagem da própria origem", texto);
    return;
}
if (!mensagem.destino().equals(tipo.name())) {
    log.descartada("destinada a outro processo", texto);
    return;
}
if (mensagem.tipo() != TipoMensagem.REQ) {
    log.descartada("tipo incompatível", texto);
    return;
}
pool.submit(() -> processar(mensagem));
```

3.3 Concorrência e estado compartilhado

O laço receptor não calcula evidências: ele entrega cada requisição válida a um `ExecutorService` fixo. Um conjunto concorrente protege os UUIDs já atendidos, evitando processamento repetido dentro da mesma instância.

```
private final Set<String> avaliacoesRecebidas =
        ConcurrentHashMap.newKeySet();
private final ExecutorService pool =
        Executors.newFixedThreadPool(config.threads());

if (!avaliacoesRecebidas.add(mensagem.id())) {
    log.descartada("requisição duplicada", texto);
    return;
}
pool.submit(() -> processar(mensagem));
```

3.4 Timeout, correlação e duplicidade

O coordenador mantém um `ConcurrentMap<UUID, AvaliacaoEmAndamento>`. Cada contexto contém mapas indexados por tipo de agente e um `CountDownLatch(4)`. Uma segunda resposta do mesmo agente não substitui a primeira. Após o prazo configurável, agentes sem resposta são listados e a LPA2v não é executada.

```
if (respostas.containsKey(agente) || erros.containsKey(agente)) {
    log.descartada("resposta duplicada", mensagem.serializar());
    return;
}
respostas.put(agente, mu);
pendentes.countDown();

pendentes.await(timeoutMs, TimeUnit.MILLISECONDS);
ausentes.removeAll(respostas.keySet());
ausentes.removeAll(erros.keySet());
```

3.5 Cálculo das evidências

Tabagismo soma as cinco alternativas válidas e divide por 13; sedentarismo divide por 9. Pressão usa o maior valor entre `(PAS-120)/60` e `(PAD-80)/30`. O IMC é `peso/altura²` e sua evidência é `(IMC-25)/15`. Pressão e IMC são limitados a `[0,1]`. O IMC ignora baixo peso, uma limitação consciente da escala.

```
double evidenciaPas = (pas - 120.0) / 60.0;
double evidenciaPad = (pad - 80.0) / 30.0;
return limitar(Math.max(evidenciaPas, evidenciaPad));

double imc = peso / (altura * altura);
return limitar((imc - 25.0) / 15.0);
```

Os agentes recusam quantidade errada, alternativa inexistente, número não finito, pressão/peso não positivos e altura fora de 0,50 a 2,50 m. Nesse caso enviam `ERRO` com o mesmo UUID.

O valor trafegado preserva a precisão do `double` com ponto decimal e `Locale.US`. O arredondamento para três casas ocorre somente nas tabelas apresentadas pelo coordenador, evitando propagar erro entre os NAPs.

3.6 Rede LPA2v

Para cada nó, `Gc=μ-λ`, `Gct=μ+λ-1` e `μE=(Gc+1)/2`. O NAP1 recebe `μT` e `1-μS`; o NAP2 recebe `μPA` e `1-μIMC`; o NAP3 recebe `μE1` e `1-μE2`. A classificação testa, nesta ordem, `Gc>=0,5`, `Gc<=-0,5`, `Gct>=0,5` e `Gct<=-0,5`; caso contrário, o estado é não extremo.

```
NoLpa2v nap1 = calcular("NAP1", tabagismo, 1 - sedentarismo);
NoLpa2v nap2 = calcular("NAP2", pressao, 1 - imc);
NoLpa2v nap3 = calcular("NAP3", nap1.evidenciaResultante(),
        1 - nap2.evidenciaResultante());
```

4. Decisões de projeto e alternativas descartadas

- Protocolo textual foi escolhido por facilitar inspeção dos datagramas e logs; serialização Java foi descartada por acoplar o protocolo às classes.
- Um único grupo atende os dois sentidos e respeita o enunciado. TCP e UDP unicast foram descartados porque criariam ligações diretas.
- UUID aleatório foi preferido a contador local, pois dois coordenadores podem iniciar com o mesmo contador.
- Pool fixo limita consumo de recursos e mantém a recepção livre. Uma thread ilimitada por datagrama poderia esgotar memória sob carga.
- Erro e ausência interrompem a consolidação. Usar zero como substituto produziria uma decisão aparentemente válida baseada em dado inventado.
- Os contextos permanecem durante a execução para que respostas tardias e duplicadas continuem identificáveis no log; em serviço de longa duração seria apropriada uma expiração periódica.

5. Resultados dos testes

Os testes determinísticos em `TesteCalculosMain` verificam os seis casos com tolerância de 0,001, as classificações, altura zero, alternativa inexistente e ida e volta do protocolo. Em 29/10/2026, a compilação com OpenJDK 17 e as 45 verificações passaram. Um ensaio multicast local também executou os seis casos, confirmou o descarte de resposta duplicada e recusou altura zero. Os logs desse ensaio estão em `logs/`. A tabela esperada é:

Caso	μT	μS	μPA	μIMC	Gc final	Gct final	Estado
1	0,000	0,000	0,000	0,000	-1,000	0,000	F, risco baixo
2	1,000	1,000	1,000	1,000	1,000	0,000	V, risco elevado
3	0,615	0,778	0,500	0,336	0,115	0,279	Não extremo, elevado
4	1,000	1,000	0,000	0,000	0,000	1,000	T, inconclusivo
5	0,000	0,000	1,000	1,000	0,000	-1,000	⊥, inconclusivo
6	0,308	0,333	0,250	0,179	-0,465	0,106	Não extremo, baixo

Evidências do laboratório

Inserir aqui, após a demonstração:

1. captura e logs dos seis casos normais;
2. captura do timeout sem IMC;
3. captura do reingresso do IMC;
4. captura dos dois coordenadores simultâneos;
5. linha `resposta duplicada` com dois agentes de tabagismo;
6. mensagem `ERRO` da altura zero e avaliação incompleta.

6. Respostas às questões

6.1 Por que multicast? O que muda com um quinto agente?

Broadcast alcança indiscriminadamente toda a sub-rede e pode ser bloqueado por roteadores; quatro unicasts exigiriam quatro destinos e quatro envios diretos. Multicast permite que interessados ingressem no grupo, mantendo remetente e receptores desacoplados. Para um quinto agente, o coordenador adiciona o novo tipo/dado esperado e ajusta a consolidação e a contagem de respostas; não ganha uma nova conexão nem precisa descobrir IP individual. A mensagem continuará passando uma vez pelo grupo.

6.2 Limitações do UDP

O sistema detecta ausência por timeout, erro validado pelo agente, resposta duplicada, mensagem malformada, destino/origem inesperados e UUID desconhecido. Ele não distingue

agente desligado de requisição/resposta perdida, não recupera automaticamente datagramas e não percebe um cálculo semanticamente errado que chegue em formato válido. A ordem não afeta a correlação porque cada resposta é indexada por UUID e agente. Uma requisição perdida passa despercebida até virar ausência no timeout.

6.3 Risco de TTL maior que 1

Se roteadores multicast encaminhassem os pacotes, dois laboratórios com o mesmo grupo/porta receberiam tráfego um do outro. UUIDs reduziriam atribuição cruzada, mas agentes do outro laboratório poderiam responder às requisições e causar duplicidades, exposição de dados, carga e resultados imprevisíveis. TTL 1 limita o datagrama ao enlace local.

6.4 Cálculo manual do caso 6

$\mu T=4/13=0{,}307692$, $\mu S=3/9=0{,}333333$, $\mu PA=\max(15/60,5/30)=0{,}250000$. O IMC é $80/1{,}70^2=27{,}681661$, logo $\mu IMC=(27{,}681661-25)/15=0{,}178777$.

Nó	μ	λ	Gc	Gct	μE
NAP1	0,307692	0,666667	-0,358974	-0,025641	0,320513
NAP2	0,250000	0,821223	-0,571223	0,071223	0,214388
NAP3	0,320513	0,785612	-0,465099	0,106124	0,267450

Nenhum controle chega a ±0,5. O estado é não extremo e, como $Gc<0$, há tendência a risco baixo.

6.5 Interpretação dos casos 4 e 5

No caso 4, fatores comportamentais são máximos, mas pressão e IMC são baixos. O NAP3 chega a $Gct=1$: há polos fortes e conflitantes, portanto inconsistente. No caso 5 ocorre o inverso: comportamento sem risco e fatores clínicos máximos; a rede chega a $Gct=-1$, estado paracompleto. “Inconclusivo” é uma decisão prudente nos dois casos, mas a frase única “evidências contraditórias” é menos precisa para o paracompleto; tecnicamente seria melhor explicar que a estrutura de evidências ficou insuficiente/desalinhada para decidir.

6.6 Relação entre μE e a média

Se as entradas favoráveis são a e b, o nó usa $\mu=a$ e $\lambda=1-b$:

$\mu E = (Gc+1)/2 = (\mu-\lambda+1)/2 = (a-(1-b)+1)/2 = (a+b)/2$.

Assim, a evidência resultante coincide com a média. A LPA2v acrescenta Gc e Gct, distinguindo média produzida por concordância daquela produzida por polos conflitantes ou falta de evidência. Duas entradas podem ter a mesma média e posições diferentes no reticulado lógico, informação perdida pela média só.

6.7 Comparação com o MultiagenteCambial

Ao colocar cada agente em processo e potencialmente em máquina própria, obtêm-se isolamento de falhas, implantação independente, paralelismo real, distribuição de carga e facilidade para acrescentar participantes ao grupo. Os custos são configuração de rede, latência, protocolo explícito, concorrência, logs distribuídos, tratamento de perda/duplicidade e operação de mais processos. A versão com chamadas locais ou TCP centralizado é mais simples e confiável na entrega, mas acopla endereços e cria maior dependência do processo central.

7. Organização do grupo

O trabalho foi desenvolvido e revisado coletivamente por Evandro, Joel, Lucas, Arhur Alves e Iago. Todos devem conhecer a arquitetura, executar qualquer processo e explicar protocolo, escalas, concorrência, rede e LPA2v na demonstração.